Document number: P0880R2
Project: Programming Language C++
Audience: SG6 Numerics, Library Evolution

Igor Klevanets <cerevra@yandex.ru>, <cerevra@yandex-team.ru>
Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2019-01-15

Numbers interaction

Significant changes since P0880R1 are marked with blue.

Green lines are notes for the **editor** or for the **SG6/LEWG** that must not be treated as part of the wording.

I. Introduction and Motivation

SG6/LEWG currently works on multiple new classes that represent numbers (see P0539R2 and P0037R4 for examples). However, those new classes do not interract well with each other.

This paper attempts to solve the problem in a generic way on a library level and provide wording for interactions of different new numeric types, including user defined numeric types.

A proof of concept implementation available at: https://github.com/cerevra/int/tree/master/v3.

II. Changelog

- R2: Added a wording variant with implicilty enabled interoperability and an ability to disable it.
- R1: Added some examples and clarifications in attempt to address issues, noted by Vicente J. Botet Escriba.
- R0: Initial version (split of the P0539R2 paper)

III. Design

Imagine that we have numeric classes `A`, `B` and `C` and wish to provide wording for all the interactions of those types. In that case we'll have to describe 126 functions ((A, B, C) x (*, /, -, +, %, &, |, ^, <, >, <=, >=, ==, !=) x (A, B, C)). Such description tends to be error prone and scales badly. Addition of one more numeric class will force us to describe 224 functions.

To avoid that issue the proposal:

- defines `Arithmetic` and `Integral` concepts that require `std::numeric_limits<Arithmetic>::is_specialized` and `std::numeric_limits<Integral>::is_integer` to be true
- converts different numeric types to a single type using `common_type_t`

using the above approach we can define a all the operations for numeric classes like this:

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator*(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `common_type_t<Arithmetic1, Arithmetic2>(lhs) * common_type_t<Arithmetic1, Arithmetic2>(rhs)`.

Each paper that proposes new numeric type has to specialize `numeric_limits` and `common_type` to get the interactions.

IV. Examples and concerns

Concern: Who decides of what are the common types? If we have overflow_integer, elastic_integer, fixed_point where the common_type will be defined?

Response: Definition of the `common_type` is actually out of scope of this paper. This paper does not attempt to solve the complexities of `commmon_type` definitions.

But let's take some examples to make sure that relying on common_type helps to reduce the burden of operation definitions. Let's split the problem into two parts and take wide_integer and safe_integer as an example. First problem here is to define the `common_type_t` for the new numeric type instantiated with different template parameters. This is easy to solve in the paper that proposes the new numeric type.

In wide_integer paper we define the common type:

```
template<size_t Bits, typename S, size_t Bits2, typename S2>
struct common_type<wide_integer<Bits, S>, wide_integer<Bits2, S2>> {
  using type = wide_integer<max(Bits, Bits2), see below>;
};
```

The signed template parameter indicated by this specialization is following:

- `(is_signed_v<S> && is_signed_v<S2> ? signed : unsigned)` if `Bits == Bits2`
- `S` if `Bits > Bits2`
- `S2` otherwise

In safe_integer paper we define the common type:

```
template<class T1, class T2>
struct common_type<safe_integer<T1>, safe_integer<T2>> {
  using type = safe_integer<common_type_t<T1, T2>>;
};
```

Here comes the hard part - interactions between arbitrary Arithmetic types and new numeric type. Let's assume that wide_integer accepted first and has common_type defined in the following way:

```
template<size_t Bits, typename S, typename Arithmetic>
struct common_type<wide_integer<Bits, S>, Arithmetic> {
  using type = a lot of complex expressions;
};


template<typename Arithmetic, size_t Bits, typename S>
struct common_type<Arithmetic, wide_integer<Bits, S>>
  : common_type<wide_integer<Bits, S>, Arithmetic>;
```

Because the wide_integer paper already accepted, the safe_integer must take that into account while defining the common type and adjust the rules for wide_integer:

```
template<size_t Bits, typename S, typename Arithmetic>
struct common_type<wide_integer<Bits, S>, Arithmetic> {
  using type = a lot of complex expressions;
};
    Shuld SFINAE if Arithmetic is safe_integer.
```

... and define specialization for safe_integers:

```
template<class T, class Arithmetic>
struct common_type<safe_integer<T>, Arithmetic> {
  using type = safe_integer<common_type_t<T, Arithmetic>>;
};

template<typename Arithmetic, class T>
struct common_type<Arithmetic, safe_integer<T>>
  : common_type<safe_integer<T>, Arithmetic>;
```

Done. Now if this paper (P0880) is accepted then the operations for the above types would be available out of the box. Note that all the above manipulations are required even without this paper (P0880).

Above manipulations for defining the common type will work well with fixed_point, dyninteger, wide_integer, safe_integer, dynfloat... But fail for elastic_integer that must define the operations in some other way, uncommon to other new numeric types.

Concern: If we have overflow_integer, elastic_integer, fixed_point where the common_type will be defined?

Response: This is up to the Standard Library implementors. Probably the specializations would be all located in a single header file along with forward declarations of numeric types or some of the specialization will go into the headers of a particular numeric types directly.

Concern: Do we really wish to rely on `std::numeric_limits` and on `std::common_type` that could be specialized by user?

Response: It depends on the goals we are trying to achieve. If we also wish to provide means for simple integration of user defined types and standard library typrs - then yes, we have to rely on them. Otherwise we could define `Arithmetic` and `Integral` in terms of `is_arithmetic_v<T>` and `is_integral_v<T>` (just like Concepts TS does).

V. Proposed wording

26.2 Definitions [numerics.defns]

[Note to editor: Add the following paragraph to the end of [numerics.defns] ***- end note]***

Define `CT` as `common_type_t<A, B>`, where `A` and `B` are the types of the two function arguments.

26.3 Numeric type requirements [numeric.requirements]

[Note to editor: Add the following paragraphs to the end of [numeric.requirements] ***- end note]***

Functions that accept template parameters starting with `Arithmetic` shall not participate in overload resolution unless `std::numeric_limits<Arithmetic>::is_specialized` is `true` and `std::numeric_limits<`*`Arithmetic`*`>::is_interoperable` is `true`.

Functions that accept template parameters starting with `Integral` shall not participate in overload resolution unless `std::numeric_limits<Integral>::is_integer` is `true` and `std::numeric_limits<`*`Integral`*`>::is_interoperable` is `true`.

[Note to SG6/LEWG: Variant of the above paragraph with interoperability turned on by default: ***- end note]***

Functions that accept template parameters starting with `Arithmetic` shall not participate in overload resolution unless `std::numeric_limits<Arithmetic>::is_specialized` is `true` and `std::numeric_limits<`*`Arithmetic`*`>::is_interoperable` is **not defined or** `true`.

Functions that accept template parameters starting with `Integral` shall not participate in overload resolution unless `std::numeric_limits<Integral>::is_integer` is `true` and `std::numeric_limits<`*`Integral`*`>::is_interoperable` is **not defined or** `true`.

[Note: Only one variant should be chosen: either with expilicit enabling of interoperability or interoperability enabled by default ***- end note]***

26.4 Numeric types interoperability [numeric.interop]

Following operators are defined for types `T` and `U` if `T` and `U` have a defined `common_type_t<T, U>` and satisfy the `Arithmetic` or `Integral` requirements from [numeric.requirements] *[Note:* Implementations are encouraged to provide optimized specializations of the following operators - *end note]*:

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator*(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) * CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator/(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) / CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator+(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) + CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  common_type_t<Arithmetic1, Arithmetic2>
  constexpr operator-(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) - CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator%(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) % CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
```

```
  constexpr operator&(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) & CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator|(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) | CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, Integral2>
  constexpr operator^(const Integral1& lhs, const Integral2& rhs);
```

Returns: `CT(lhs) ^ CT(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, size_t>
  constexpr operator<<(const Integral1& lhs, const Integral2& rhs);
```

Returns: `common_type_t<Integral1, size_t>(lhs) << static_cast<size_t>(rhs)`.

```
template<typename Integral1, typename Integral2>
  common_type_t<Integral1, size_t>
  constexpr operator>>(const Integral1& lhs, const Integral2& rhs);
```

Returns: `common_type_t<Integral1, size_t>(lhs) >> static_cast<size_t>(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator<(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) < CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator>(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) > CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator<=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) <= CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator>=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) >= CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator==(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `CT(lhs) == CT(rhs)`.

```
template<typename Arithmetic1, typename Arithmetic2>
  constexpr bool operator!=(const Arithmetic1& lhs, const Arithmetic2& rhs);
```

Returns: `!(lhs == rhs)`.

VI. Feature-testing macro

For the purposes of SG10 we recommend the feature-testing macro name `__cpp_lib_num_interop`.